Solution Architecture

Date	5 July 2026
Team ID	
Project Name	Rising Waters
Maximum Marks	5 Marks

Solution Architecture Diagram:
Provide a high-level visual representation of your solution's architecture. Use the structural layout below as a template to map out your Presentation Layer, Application Layer, and Data Layer. Fill in the specific technologies and components for your project directly into the boxes to create your architectural diagram.

Presentation / Client Layer
Flood Prediction Web Application (HTML, CSS, JavaScript)

↓↑

API Gateway / Load Balancer
Python Anywhere Web Server

↓↑

Auth Service	**Core Logic Service**	**External API Integrations**
Input Validation (Flask Backend)	*Pandas, Data Prep, Random Forest Model (Prediction Engine)*	*Python Anywhere Deployment API*

↓↑

Data / Storage Layer
SQLite database, Model Files (floods.save & scaler.save)

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Renders the user facing web screens. It captures user inputs for the 8 weather parameters (Temperature, Humidity, Cloud Cover, Annual rainfall, and seasonal totals) and displays prediction results (warning/safety pages), dynamic history logs and data analysis visualizations.	HTML5, CSS3, JavaScript
API Gateway	Acts as the entry point for incoming HTTP traffic, routing web requests to the appropriate backend application logic and serving static files (CSS, images) back to the client.	PythonAnywhere Routing
Core Logic Services	Receives weather inputs, standardizes the metrics using a pre-calculated mathematical scale and inputs them into a saved statistical classification model to output the final risk percentage and prediction.	Python 3, Flask, Joblib, NumPy, Pandas, Scikit-learn (StandardScaler)
Datalayer	Persists user credentials and keeps chronological records of all historical predictions, allowing users to view their previous evaluations. Also stores the pre-trained classification model parameters.	SQLite3 (database.db), local file system storage (floods.save, scaler.save)
External API Integrations	Automates administrative task deployment by programmatically uploading local updates, writing configuration files, installing requirements and reloading the application.	PythonAnywhere REST API (used in deploy.py), Requests
Auth Service	Restricts access to forecasting pages, registers user accounts securely, verifies passwords using cryptography and maintains active login states via encrypted session tokens.	Security (check_password_hash, generate_password_hash), Flask Sessions